

QUESTION 1

```
from collections import deque
```

```
def solve_water_jug():
    # Initial states and capacities
    # (jug4, jug3)
    initial_state = (0, 0)
    goal_jug4 = 2

    # Queue for BFS stores: (current_state, path_taken)
    queue = deque([(initial_state, [initial_state])])
    visited = set([initial_state])

    print("Searching for Water Jug solution path...")

    while queue:
        (j4, j3), path = queue.popleft()

        # Goal check
        if j4 == goal_jug4:
            print(f"\nGoal Reached! Steps taken ({len(path)-1}):")
            for i, state in enumerate(path):
                print(f"Step {i}: 4-Gal Jug = {state[0]}g, 3-Gal Jug = {state[1]}g")
            return

        # Define all 8 possible rules/actions
        moves = [
            (4, j3),          # Fill 4-gal jug
            (j4, 3),          # Fill 3-gal jug
            (0, j3),          # Empty 4-gal jug
            (j4, 0),          # Empty 3-gal jug
            (4, j3 - (4 - j4)) if j4 + j3 >= 4 else (j4 + j3, 0), # Pour 3 -> 4
            (j4 - (3 - j3), 3) if j4 + j3 >= 3 else (0, j4 + j3) # Pour 4 -> 3
        ]

        for next_state in moves:
            # Ensure quantities stay within physical bounds
            next_j4, next_j3 = next_state
            if 0 <= next_j4 <= 4 and 0 <= next_j3 <= 3:
                if next_state not in visited:
                    visited.add(next_state)
                    queue.append((next_state, path + [next_state]))

if __name__ == "__main__":
```

```
solve_water_jug()
QUESTION 2
```

```
import heapq
```

```
def uniform_cost_search(graph, start, goal):
    # Priority Queue stores tuples of: (cumulative_cost, current_node, path_taken)
    # It automatically sorts by the lowest cumulative cost
    priority_queue = [(0, start, [start])]
    visited = set()

    print(f"Starting Uniform Cost Search from {start} to {goal}...\n")

    while priority_queue:
        # Pop the node with the lowest path cost
        cost, current_node, path = heapq.heappop(priority_queue)

        print(f"Visiting node '{current_node}' with current path cost: {cost}")

        # Goal check
        if current_node == goal:
            print("\n" + "="*40)
            print(f"OPTIMAL PATH FOUND: {' -> '.join(path)}")
            print(f"TOTAL MINIMUM COST: {cost}")
            print("="*40)
            return path, cost

        if current_node not in visited:
            visited.add(current_node)

            # Explore neighbors
            for neighbor, edge_cost in graph.get(current_node, {}).items():
                if neighbor not in visited:
                    total_cost = cost + edge_cost
                    heapq.heappush(priority_queue, (total_cost, neighbor, path + [neighbor]))

    return None, float('inf')

# Define the graph from the assessment diagram
delivery_network = {
    'S': {'A': 1, 'G': 12},
    'A': {'B': 3, 'C': 1},
    'B': {'D': 3},
    'C': {'D': 1, 'G': 2},
```

```
'D': {'G': 3},  
'G': {}  
}
```

```
if __name__ == "__main__":  
    optimal_path, min_cost = uniform_cost_search(delivery_network, 'S', 'G')
```

QUESTION 3

```
def is_safe(board, row, col):  
    # Check this row on left side  
    for i in range(col):  
        if board[row][i] == 1:  
            return False  
  
    # Check upper diagonal on left side  
    for i, j in zip(range(row, -1, -1), range(col, -1, -1)):  
        if board[i][j] == 1:  
            return False  
  
    # Check lower diagonal on left side  
    for i, j in zip(range(row, 8, 1), range(col, -1, -1)):  
        if board[i][j] == 1:  
            return False  
  
    return True  
  
def solve_8_queens_util(board, col):  
    # Base case: If all queens are placed  
    if col >= 8:  
        return True  
  
    # Consider this column and try placing this queen in all rows one by one  
    for i in range(8):  
        if is_safe(board, i, col):  
            board[i][col] = 1 # Place queen  
  
            # Recur to place rest of the queens  
            if solve_8_queens_util(board, col + 1):  
                return True  
  
            board[i][col] = 0 # Backtrack if placing queen doesn't lead to a solution  
  
    return False
```

```

def print_board(board):
    print("Found Valid 8-Queens Layout:")
    for row in board:
        print(" ".join("Q" if x == 1 else "." for x in row))

if __name__ == "__main__":
    chess_board = [[0] * 8 for _ in range(8)]
    if solve_8_queens_util(chess_board, 0):
        print_board(chess_board)
    else:
        print("Solution does not exist")

```

QUESTION 4

```
import math
```

```

class CabAgent:
    def __init__(self, current_location):
        self.current_location = current_location

    def get_active_drivers(self):
        # Simulated driver pool: (Name, Location_X, Location_Y, Cab_Type)
        return [
            {"name": "Driver_1", "x": 2, "y": 3, "type": "mini"},
            {"name": "Driver_2", "x": 8, "y": 9, "type": "sedan"},
            {"name": "Driver_3", "x": 1, "y": 1, "type": "prime"},
            {"name": "Driver_4", "x": 5, "y": 4, "type": "mini"},
        ]

    def solve_cab_routing(self, destination, user_preference):
        print(f"Goal: Route from {self.current_location} to {destination} ({user_preference})")

        # 1. Filter by cab comfort type
        drivers = self.get_active_drivers()
        eligible_drivers = [d for d in drivers if d["type"] == user_preference.lower()]

        if not eligible_drivers:
            return "Failure: No matching cab types available nearby."

        # 2. Select closest driver (Minimum ETA)
        selected_driver = None
        min_distance = float('inf')

        for d in eligible_drivers:

```

```

dist = math.sqrt((d["x"] - self.current_location[0])**2 + (d["y"] - self.current_location[1])**2)
if dist < min_distance:
    min_distance = dist
    selected_driver = d

```

3. Formulate Action Sequence path

```

path_sequence = f"Route Map: Start at {self.current_location} -> Dynamic Traffic
Waypoints -> Arrive at {destination}"

```

```

print(f"Matched with {selected_driver['name']} (Distance: {min_distance:.2f} units away)")
return f"Action Sequence: [Dispatch {selected_driver['name']}] -> [Execute navigation:
{path_sequence}]"

```

```

if __name__ == "__main__":
    # User is at (0,0), wants to go to (10,10) via a 'mini' cab
    ola_agent = CabAgent(current_location=(0, 0))
    result = ola_agent.solve_cab_routing(destination=(10, 10), user_preference="mini")
    print(result)

```

QUESTION 5

```
import heapq
```

```

def uniform_cost_search(graph, start, goal):
    # Priority Queue stores tuples of: (cumulative_cost, current_node, path_taken)
    # It automatically sorts by the lowest cumulative cost (Priority Queue logic)
    priority_queue = [(0, start, [start])]
    visited = set()

```

```

print(f"Starting Uniform Cost Search from {start} to {goal}...\n")

```

```

while priority_queue:
    # Pop the node path with the lowest cumulative cost
    cost, current_node, path = heapq.heappop(priority_queue)

```

```

print(f"Visiting node '{current_node}' with current path cost: {cost}")

```

```

# Goal check: If we pop the goal node, this path is guaranteed to be optimal
if current_node == goal:
    print("\n" + "="*40)
    print(f"OPTIMAL PATH FOUND: {' -> '.join(path)}")
    print(f"TOTAL MINIMUM COST: {cost}")
    print("="*40)
    return path, cost

```

```

if current_node not in visited:
    visited.add(current_node)

    # Explore all neighbors of the current node
    for neighbor, edge_cost in graph.get(current_node, {}).items():
        if neighbor not in visited:
            total_cost = cost + edge_cost
            heapq.heappush(priority_queue, (total_cost, neighbor, path + [neighbor]))

return None, float('inf')

# Graph representation from the assessment diagram
delivery_network = {
    'S': {'A': 1, 'G': 12},
    'A': {'B': 3, 'C': 1},
    'B': {'D': 3},
    'C': {'D': 1, 'G': 2},
    'D': {'G': 3},
    'G': {}
}

if __name__ == "__main__":
    optimal_path, min_cost = uniform_cost_search(delivery_network, 'S', 'G')

```

